

Homework-3: NoSQL Databases and Data Streaming

Deadline: September 25th, 11:59PM ET.

In this homework, you will practice Neo4J and Apache Kafka.

Your homework submission should be on GitHub. Use the following GitHub classroom to access the assignment: <https://classroom.github.com/a/vZRFAqXV>

- **Don't create a public repository in your own GitHub account. Use the URL above to create the GitHub repository for this assignment. It includes a starter ReadMe file.**
- On Gradescope, only submit a URL to your GitHub Classroom repository.
- **All screenshots must include your name (or a strong indicator for your identity).**
 - **If your output is not printed in a Jupyter notebook, take a screenshot of the output and make sure your name is written in a Text Editor in the screenshot (Don't photoshop your name).**
- Not following the submission guidelines may result in grading penalties.

Cite any external sources you use. External sources shouldn't exceed more than 30% of the final solution.

Submit the HW in one or more Jupyter notebooks (or a series of python files with clear comments). If you need to include scripts outside Jupyter for a given question, note it in your notebook. If you need to include output or screenshots, you may include them in your notebook or attach them as separate image files using the naming convention qx.png (where x is the question number). If you have more than one image for a given question, name the files as qx_1.png, qx_2.png, etc.

A sample structure of your notebook is shown below

Q1

Refer to q1.sql and q1.png

Q2

```
from pyspark.sql.functions import monotonically_increasing_id

### Some code here
## For output, refer to q2.png
```

We will use the Shopping Trends Dataset. You can access it from this URL:
https://www.andrew.cmu.edu/user/mfarag/static/shopping_trends.csv

Use the most scalable possible technique to answer each of the following questions.

Q1. (10%): Load the dataset into Spark and display the descriptive statistics associated with the dataset (e.g., mean, standard deviation, etc.).

- Submit both code and output screenshot of the descriptive statistics.

Q2. (30%): Graph databases excel in powering recommendation engines. Your task is to design and implement a Python script to store a dataset of user purchases in Neo4j, creating an intuitive, scalable model for analyzing relationships between users and their purchased items. Consider the following key requirements in your design:

- Capture user attributes like age and location for future analysis.
- Efficiently store items with unique nodes, ensuring each item (e.g., "Blouse" or "Sweater") appears only once in the graph.
- Add purchase-specific details directly to the relationships between users and items.
- Programmatically insert all 3900 user-item relationships without hard-coding them.
- Provide both your Python code and a screenshot of a query that retrieves the stored records.

At this point, assume you no longer have access to the CSV file and use your graph store to answer the questions below.

Q3. (15%): Develop a python function to find the overall percentage of users who are 40+ years old in the dataset.

- Submit both code and output screenshot.

Q4. (15%): Develop a python function to find the most purchased item in California.

- Submit both code and output screenshot.
- If multiple items are returned in the results, display them all.
- Hint: consider using several functions at Neo4J. You may find the following resources useful:
 - Count: <https://neo4j.com/docs/cypher-manual/current/functions/aggregating/>
 - Limit: <https://neo4j.com/docs/cypher-manual/current/clauses/limit/>

Q5. (10%): What is the most popular season to shop in? and what is the most popular shipping method in this season?

- Note: you can ignore the frequency of purchases from your calculations/assumptions.
- Submit both code and output screenshot.

Q6. (20%): YouTube Video Analysis Based on Comment Popularity:

Enhance the YouTube demonstration from the lecture by developing a feature that identifies the most popular video out of up to five videos. This will be determined by calculating which video has the highest total number of likes on its comments.

Design your application to prompt the user to input up to five YouTube video IDs.

Ensure your application aggregates the total number of likes from the comments on each video and displays the title of the video that has accumulated the most likes on its comments.

You may assume that calling the YouTube API once is sufficient for retrieving all the comments that you need to consider for your analysis.

Ensure that all communications with the YouTube API are confined to the Producer script to avoid redundant API calls from other areas of the code.

You can print the output on a terminal shell or a webpage.

Your application must use Apache Kafka (or Confluent Kafka).

Hint: You may find the YouTube API reference useful:

<https://developers.google.com/youtube/v3/docs/>

For this question, submit the following:

- (10%) All the code files you developed/used in a working state.
- (5%) Screenshot of the “Messages” tab of your Kafka topic with sample messages from your Kafka topic.
 - Hint: open the “Messages” tab before you start the Producer to view the messages properly.
- (5%) Screenshot of the output of the consumer.